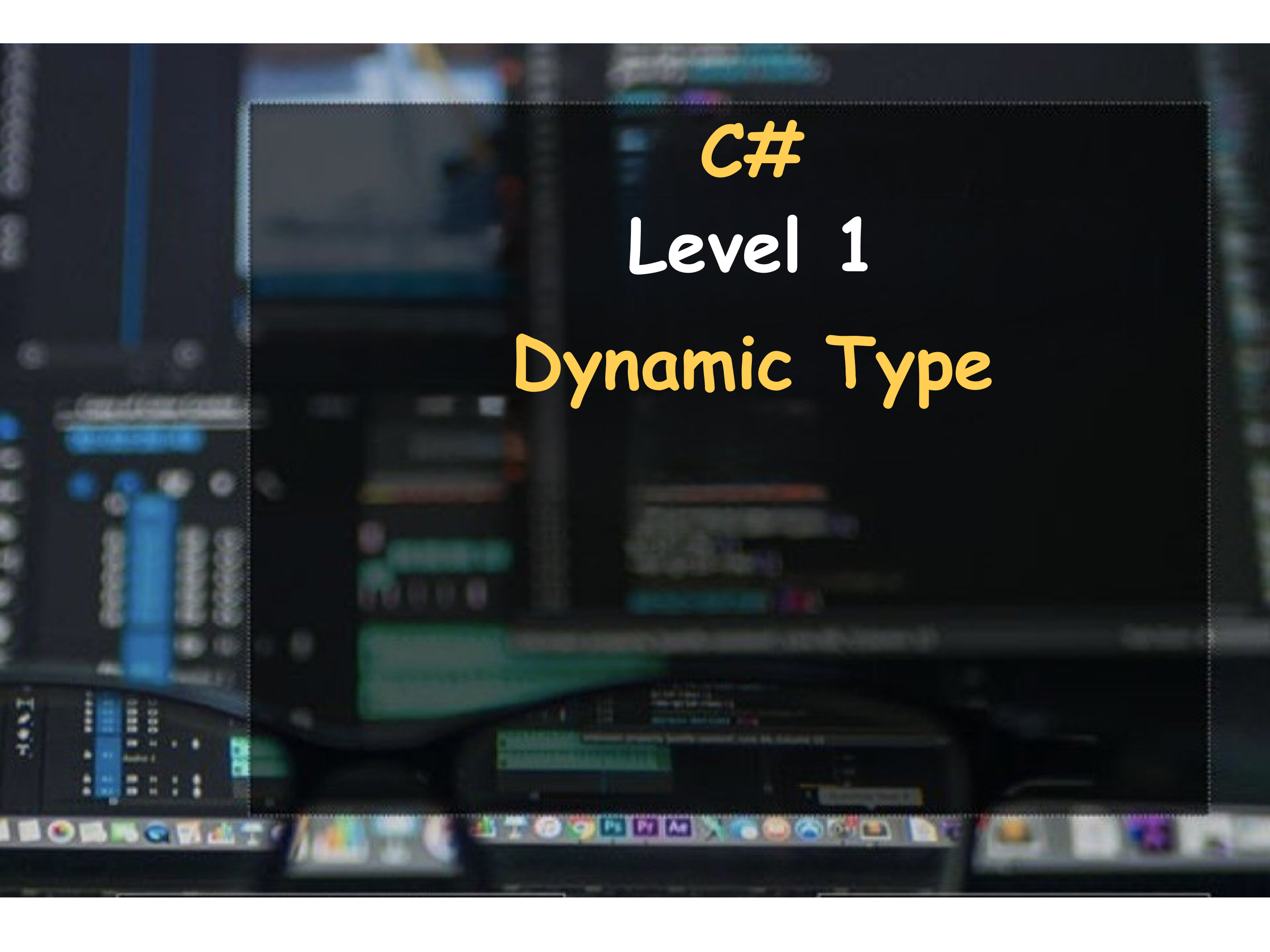
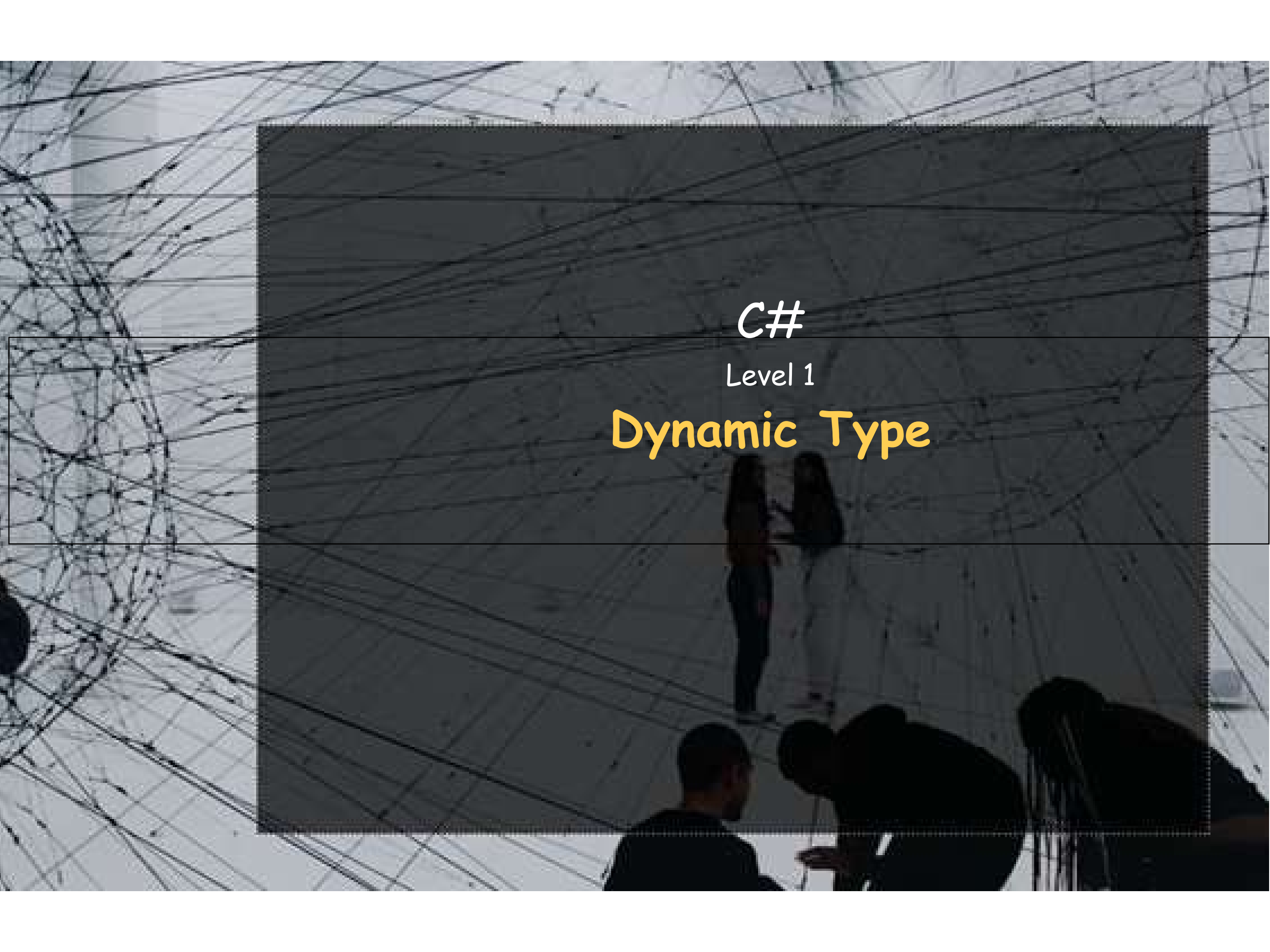




C#

Level 1

Dynamic Type



C#

Level 1

Dynamic Type

Definition

Dynamic Type

-  `dynamic` bypasses compile-time type checking.
-  The actual type of the variable is determined at runtime.

Explanation

Storing Different Types

-  A `dynamic` variable can store values of different types during program execution.

-  The variable can be reassigned to another type without changing its declaration.

Procedures

1. Declare the variable using ``dynamic``.
2. Assign a value of any type.
3. Reassign a value of another type if needed.
4. Operations are checked at runtime.
5. Use ``GetType()`` to inspect the current runtime type.

💡 Example

```
dynamic MyDynamicVar = 100;  
Console.WriteLine(  
    "Value: {0}, Type: {1}",  
    MyDynamicVar,  
    MyDynamicVar.GetType()  
);
```

➡ Result

Value: 100, Type: System.Int32

Note

- ◆ `dynamic` delays type checking until runtime.
- ◆ The variable itself does not have to keep the same type.
- ◆ The actual value still has a real .NET type at runtime.

⊘ Warning

◆ Errors that would normally be detected at compile time may only appear when the code runs.



Comparison

`var`

- ✓ Type is determined by the compiler.
- ✓ The variable remains strongly typed.
- ✓ Compile-time type checking is still applied.

`dynamic`

- ✓ Type checking is performed at runtime.
- ✓ The variable can hold different types during execution.
- ✓ Some type errors are detected only when the program runs.

✦ Summary

- ✓ ``dynamic`` bypasses compile-time type checking.
- ✓ Its actual type is determined at runtime.
- ✓ A ``dynamic`` variable can store different types.
- ✓ ``GetType()`` shows the current runtime type.
- ✓ ``var`` is strongly typed after compilation.
- ✓ ``dynamic`` provides more flexibility but can move errors from compile time to runtime.

Thank you 😊